

DEPARTAMENTO
DE COMPUTACION

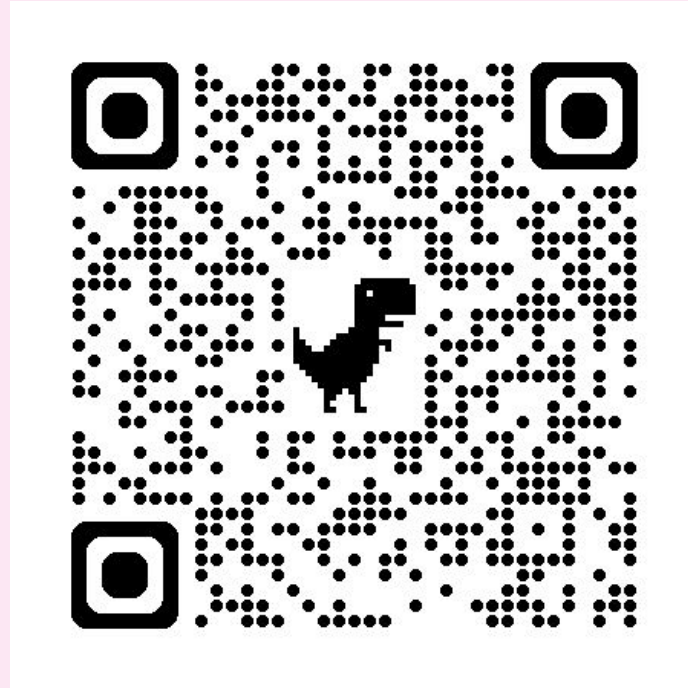
Facultad de Ciencias Exactas y Naturales - UBA

Construyendo Agentes de Inteligencia Artificial

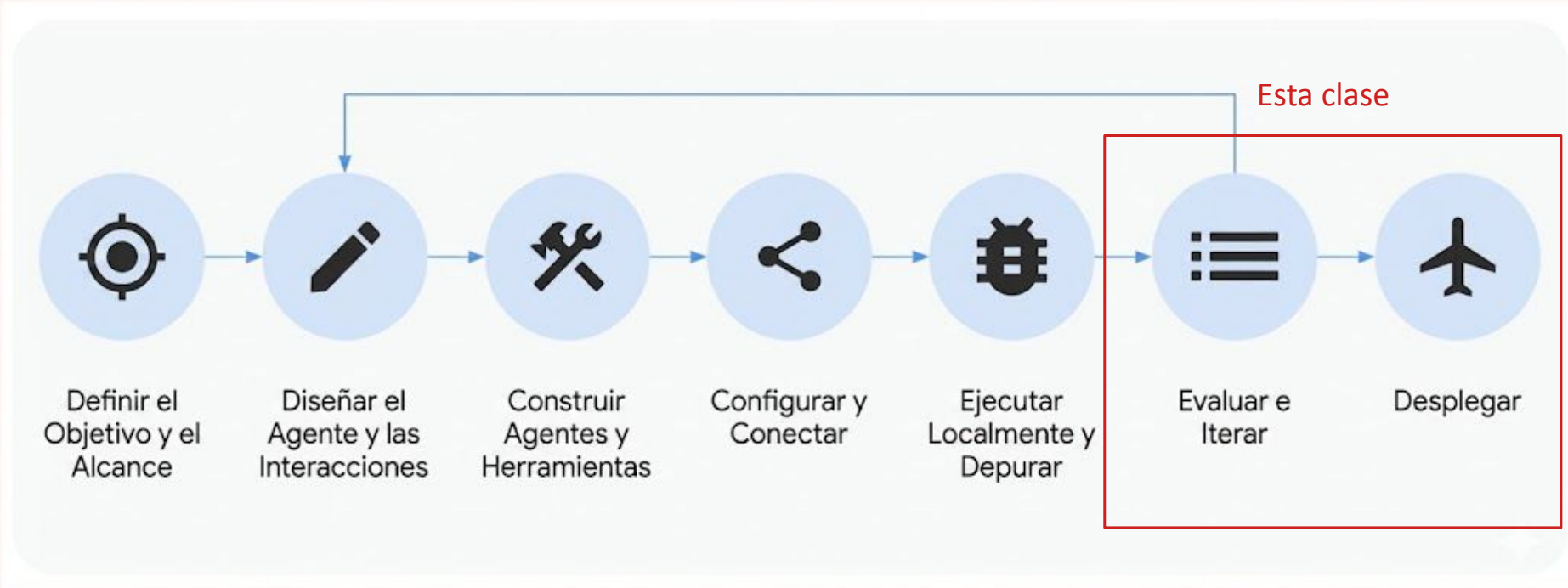
Día 5: Multi-agentes, Evaluación y Futuro

Viernes 31 de Julio del 2026

Repaso Clase 4



Proceso de Creación de Agentes





Avaluar e Iterar

Probar los sistemas de ML

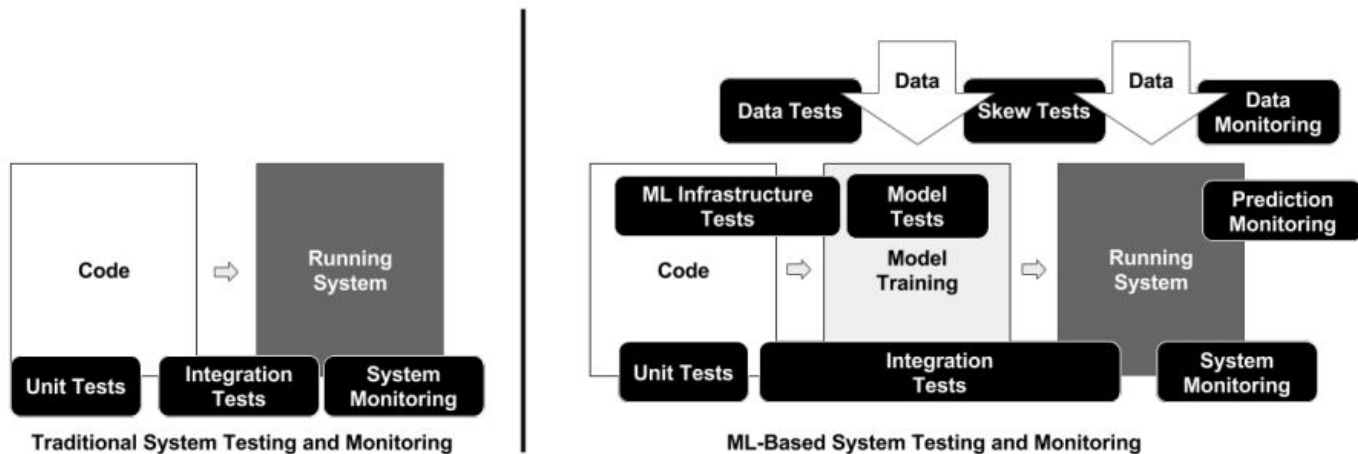


Figure 1. **ML Systems Require Extensive Testing and Monitoring.** The key consideration is that unlike a manually coded system (left), ML-based system behavior is not easily specified in advance. This behavior depends on dynamic qualities of the data, and on various model configuration choices.

Los modelos requieren una combinación de testeo al momento de la creación / desarrollo / implementación del sistema de ML (offline evaluation) y evaluaciones / monitoreos al momento de estar en producción (online evaluation)

Evaluación de Agentes

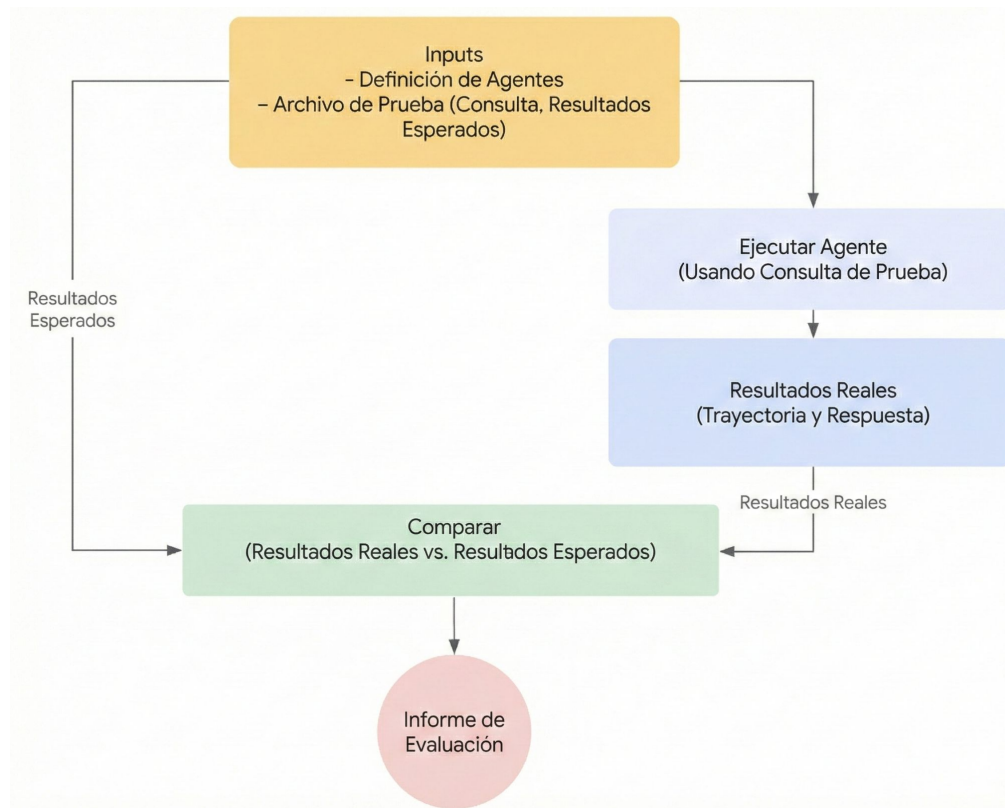
Evaluar los agentes es **fundamental**. Tienen muchísima dificultad porque hay que considerar las distintas entradas posibles que escribirán los usuarios y estas pueden ser infinitas.

Además, por la naturaleza **no determinística** de los agentes, las respuestas que estos generen pueden variar.

Considerar además, las herramientas que podrían tener distintos comportamientos, incrementando la variabilidad.

Probar un Agente Conversacional tiene muchísima dificultad y criticidad. Se recomienda observar:

- Documentar varios casos de pruebas, en el orden de los miles.
- Usar LLMs para generar casos de prueba y/o testear iterativamente el agente.
- Encapsular pruebas propias de las herramientas midiendo respuestas y tiempos de latencia.
- Documentar las pruebas en el entorno QA
- Al deployar, hacerlo gradual y observar las conversaciones de un pequeño porcentaje de Agentes para ver sus riesgos
- Utilizar framework específicos



ADK Criterios

Criterio	Descripción	Basado en Referencia	Requiere Rúbricas	LLM como Juez	Soporta Simulación de Usuario
tool_trajectory_avg_score	Coincidencia exacta de la trayectoria de llamadas a herramientas	Si	No	No	No
response_match_score	Similitud ROUGE-1 con respuesta de referencia	Si	No	No	No
final_response_match_v2	Coincidencia semántica juzgada por LLM con respuesta de referencia	Si	No	Si	No
rubric_based_final_response_quality_v1	Calidad de respuesta final juzgada por LLM basada en rúbricas personalizadas	No	Si	Si	Si
rubric_based_tool_use_quality_v1	Calidad de uso de herramientas juzgada por LLM basada en rúbricas personalizadas	No	Si	Si	Si
hallucinations_v1	Fundamentación de la respuesta del agente juzgada por LLM contra el contexto	No	No	Si	Si
safety_v1	Seguridad/inofensividad de la respuesta del agente	No	No	Si	Si
per_turn_user_simulator_quality_v1	Calidad del simulador de usuario juzgada por LLM	No	No	Si	Si

Notebook

Evaluación de Agentes en ADK

LLM-as-a-Judge

Con la evolución de los LLMs hacia asistentes conversacionales avanzados, los *benchmarks* estáticos de opción múltiple ya no logran medir su rendimiento en tareas abiertas. Al mismo tiempo, depender exclusivamente de la evaluación humana resulta demasiado lento, costoso y difícil de escalar frente a textos generados cada vez más sofisticados y difíciles de distinguir.

LLM-as-a-judge : Para resolver este cuello de botella, surge el enfoque de utilizar modelos de última generación (como GPT-4) para evaluar automáticamente a otros modelos. Al darle al "LLM Juez" los criterios y las respuestas a comparar, este método logra coincidir en más de un 80% con las preferencias humanas (el mismo nivel de acuerdo que hay entre dos personas), ofreciendo una alternativa rápida, económica y explicable.

A pesar de su eficiencia, delegar la evaluación a una IA requiere un diseño cuidadoso porque introduce nuevos riesgos. Es fundamental que los desarrolladores identifiquen y mitiguen fallas como:

- el **sesgo de posición** (favorecer la primera respuesta que lee),
- el **sesgo de verbosidad** (premiar respuestas largas),
- el **sesgo de automejora** (beneficiar a modelos de su propia familia)
- sus propios límites lógicos ante el razonamiento profundo.

Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena <https://arxiv.org/abs/2306.05685>

<https://www.lmsys.org/blog/2023-03-30-vicuna/>

Ejemplo de uso de LLM-as-Judge en Investigación

Few-shot ICL pipeline

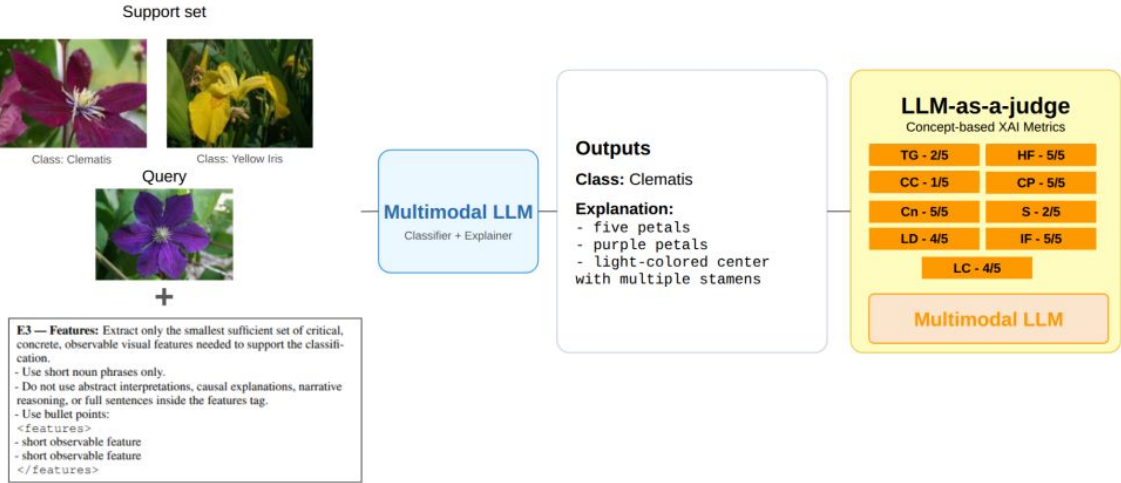


Figure 1. Overview of the In-Context Learning Explain-Classify and Evaluate pipeline. The Multimodal LLM acts as both classifier and explainer, while a separate LLM-as-a-judge evaluates nine concept-based XAI metrics (1–5 ↑): see Table 1. The example shown corresponds to E3 (feature-based explanation), where the model is prompted to identify the minimal sufficient set of critical, concrete, and observable visual features needed to support the classification. The judge, using a different MLLM and prompt, scores the generated explanation across the same nine evaluation dimensions.

3. Concept Counting

Desideratum: The explanation accurately quantifies the exact number of relevant concepts present in the image.

Definition: A precisely counted explanation ensures that the elements counted in the NLE faithfully correspond with the number of concepts in the image.

What it measures: Evaluates the quantitative accuracy of the explanation (penalizing any discrepancies between the stated count of elements in the text and the actual number of countable elements visible in the image).

Note: This metric is different from hallucination-free because it does reflect, for instance, a model explaining a flower of class f has 5 petals while the query image shows 6 petals.

Scores:

- **Wrong concept counting:** 0% of the countable features are well counted.
- **Weak concept counting:** 1–33% of the countable features are well counted.
- **Moderate concept counting:** 34–66% of the countable features are well counted.
- **Almost perfect concept counting:** 67–99% of the countable features are well counted.
- **Perfect concept counting:** 100% of the countable features are well counted.

Explaining is Harder Than Predicting Alone: Evaluating Concept-based Explanations of MLLMs as ICL Visual Classifiers

<https://arxiv.org/abs/2605.28215>

Ejemplo de uso de LLM-as-Judge en Investigación

HITL Evaluator CARMEN — Item 92/128 128 / 128 completed 100.0% [Export CSV](#)

Item 92 / 128 Incorrect



Query image · pets · index 156

Condition	E5 — axioms_ontology_v2
Model	Gemini-2.5-Flash
Dataset	pets
Config	4-way 1-shot
Expected	Ragdoll
Predicted	Maine Coon

Model Explanation

```
<tbox>
- [19] ⊆ hasVisualFeature(Dog) ∧ hasVisualFeature(BrownFur) ∧ hasVisualFeature(OutdoorSetting)
- [32] ⊆ hasVisualFeature(Cat) ∧ hasVisualFeature(SiamesePattern) ∧ hasVisualFeature(BlueEyes) ∧ hasVisualFeature(IndoorSetting)
- [26] ⊆ hasVisualFeature(Cat) ∧ hasVisualFeature(OpenMouth) ∧ hasVisualFeature(WhiteAndGreyFur) ∧ hasVisualFeature(IndoorSetting)
- [20] ⊆ hasVisualFeature(Cat) ∧ hasVisualFeature(LongHair) ∧ hasVisualFeature(BrownAndWhiteFur) ∧ hasVisualFeature(IndoorSetting) ∧ hasVisualFeature(YellowBowl)
</tbox>
<abox>
- hasVisualFeature(Query, Cat)
- hasVisualFeature(Query, LongHair)
- hasVisualFeature(Query, BrownAndWhiteFur)
- hasVisualFeature(Query, IndoorSetting)
</abox>
<dl_explanation>
- [20] ⊆ hasVisualFeature(Cat) ∧ hasVisualFeature(LongHair) ∧ hasVisualFeature(BrownAndWhiteFur) ∧ hasVisualFeature(IndoorSetting) ∧ hasVisualFeature(YellowBowl)
</dl_explanation>
<response>20</response>
```

Support images



[19] Leonberger [32] Siamese [26] Ragdoll [20] Maine Coon

1. Local Discriminateness (LD)

1 2 3 4 5

Local Discriminateness (LD)
Assesses whether the explanation highlights features that distinguish the predicted class from the other options in the episode.

1 — Non-discriminative: 0% pathognomonic features
2 — Weakly discriminative: 1–33% pathognomonic features
3 — Moderately discriminative: 34–66% pathognomonic features
4 — Strongly discriminative: 67–99% pathognomonic features
5 — Perfectly discriminative: 100% pathognomonic features

Pathognomonic features = features that distinguish the predicted class from the other episode options.

2. Textual Groundedness (TG)

1 2 3 4 5

Textual Groundedness (TG)
Assesses the degree to which the text captures all relevant objects and attributes present in the image.

1 — Not grounded: severe omissions of key image concepts
2 — Weakly grounded: major omissions
3 — Moderately grounded: few omissions
4 — Strongly grounded: one omission
5 — Perfectly grounded: zero omissions of relevant concepts

3. Hallucination-Free (HF)

1 2 3 4 5

Hallucination-Free (HF)
Penalises the invention of features that do not exist in the image.

1 — Severe hallucinations: multiple invented claims
2 — Major hallucinations
3 — Few hallucinations
4 — One hallucination
5 — Hallucination-free: all claims are visually verifiable in the image

4. Instruction Following (IF)

1 2 3 4 5

← Previous 156 · E5/axioms_ontology_v2 · pets Next →

Explaining is Harder Than Predicting Alone: Evaluating Concept-based Explanations of MLLMs as ICL Visual Classifiers

<https://arxiv.org/abs/2605.28215>

TWA - Tolerance Weighted Agreement

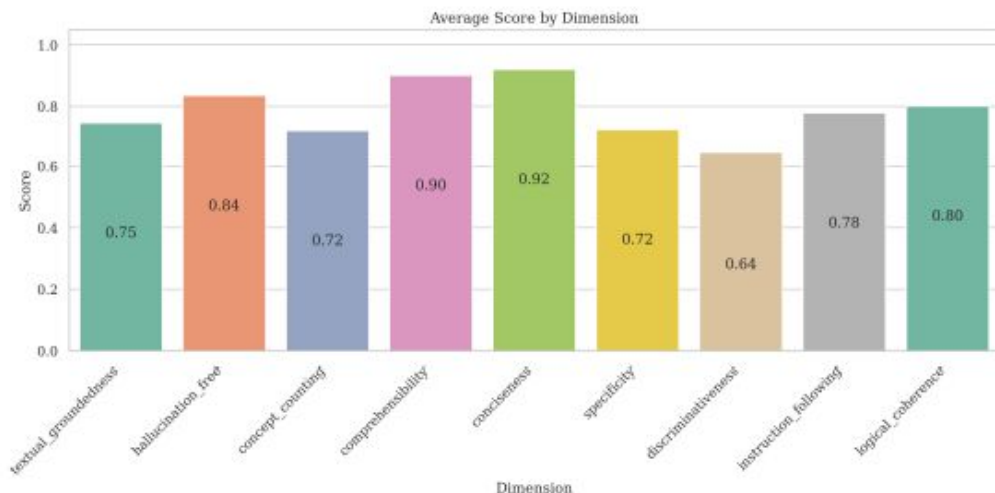


Figure 9. Average TWA scores by XAI metric. The LLM-as-a-judge demonstrates strong practical alignment on dimensions like Conciseness (0.92) and Comprehensibility (0.90). Evaluative dimensions requiring class-contrastive logic, such as Local Discriminativeness (0.64), maintain robust but comparatively lower agreement.

Let $y_{human}^{(i)}, y_{judge}^{(i)} \in \{1, 2, 3, 4, 5\}$ represent the scores assigned by the human and the algorithmic judge for the i -th evaluated instance, respectively. Defining the absolute distance between the judge and human scores as $d_i = |y_{judge}^{(i)} - y_{human}^{(i)}|$, the item-level scoring function $S(d_i)$ is formulated as:

$$S(d_i) = \begin{cases} 1 & \text{if } d_i = 0 \\ 0.75 & \text{if } d_i = 1 \\ 0 & \text{if } d_i > 1 \end{cases}$$

The overall Tolerance-Weighted Agreement (TWA) is then calculated as the average score across all N instances:

$$TWA = \frac{1}{N} \sum_{i=1}^N S(d_i)$$

Explaining is Harder Than Predicting Alone: Evaluating Concept-based Explanations of MLLMs as ICL Visual Classifiers
<https://arxiv.org/abs/2605.28215>

Uso Responsable de la IA, Gobernanza y Seguridad en Agentes

Mejores prácticas para la Prevención

Gestión de Identidad y Autorización:

- **Autenticación de Agente (Agent-Auth):** Configurar el agente para que use su propia identidad (ej. cuenta de servicio) limitando sus permisos en el sistema externo mediante políticas estrictas (como acceso de "solo lectura"). Requiere un buen registro (logs) para rastrear qué usuario desencadenó la acción.
- **Autenticación de Usuario (User Auth):** Hacer que el agente adopte la identidad del usuario activo (ej. mediante tokens OAuth), garantizando así que la IA no pueda acceder a datos o ejecutar acciones para las que el usuario humano no tiene permiso.

Implementación de Barreras de Seguridad (Guardrails):

- **Controles a nivel de Herramienta (In-tool guardrails):** Diseñar las herramientas de forma defensiva estableciendo políticas deterministas en el "Contexto de la Herramienta" (Tool Context). Estas reglas (ej. restringir las tablas de una base de datos que se pueden consultar) son fijadas por el desarrollador y no pueden ser alteradas por las decisiones del modelo.
- **Uso de Callbacks y Plugins:** Validar las entradas y salidas antes o después de la ejecución de una herramienta. Se recomienda usar modelos rápidos y económicos (como Gemini Flash Lite) para analizar y filtrar *prompts* y respuestas.
- **Filtros nativos del modelo:** Aprovechar características de seguridad integradas en modelos como Gemini, utilizando filtros de contenido e instrucciones del sistema robustas para guiar el comportamiento.

Aislamiento de la Ejecución (Sandboxing):

- Cualquier código generado por el modelo debe ejecutarse siempre en entornos seguros y aislados (sandboxed) para evitar que comprometan la seguridad de la infraestructura subyacente.

Mejores prácticas para la Prevención

Controles de Red Perimetrales:

- Confinar la actividad del agente dentro de perímetros de red seguros (como los VPC Service Controls de Google Cloud) para evitar la filtración o extracción maliciosa de datos (data exfiltration) y reducir el radio de impacto en caso de incidentes.

Evaluación y Trazabilidad (Observabilidad):

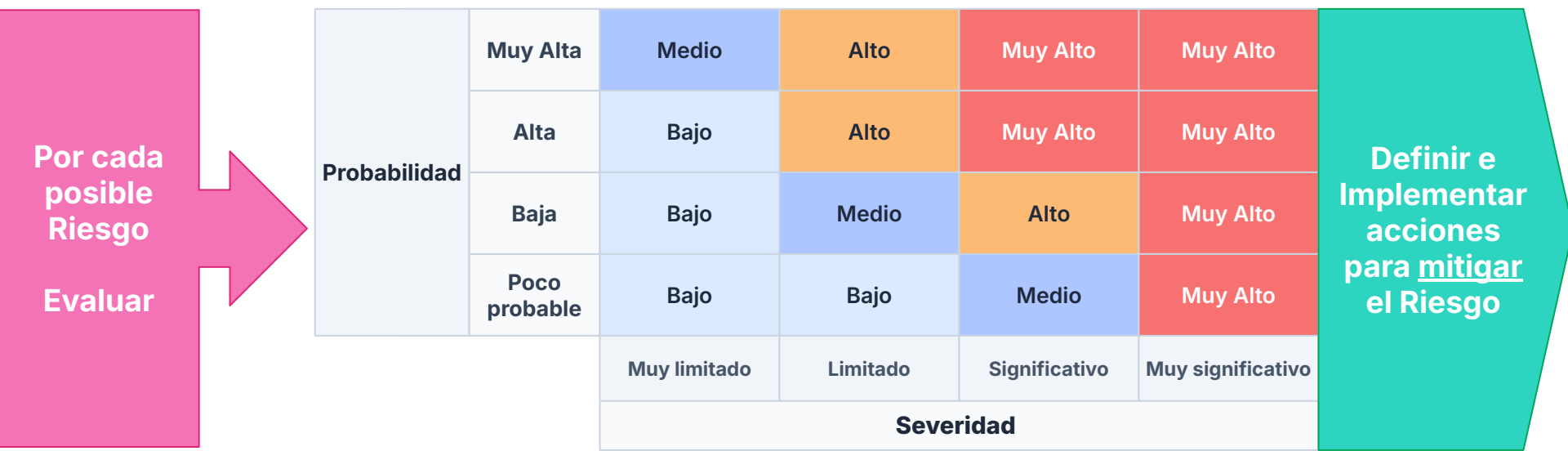
- Mantener una supervisión constante mediante herramientas que evalúen la precisión y calidad de las salidas.
- Implementar trazas (tracing) para tener visibilidad absoluta sobre el "razonamiento" del agente, las herramientas que decidió utilizar y las estrategias que aplicó.

Seguridad en la Interfaz de Usuario:

- Escapar siempre todo el contenido generado por el modelo (Always Escape Model-Generated Content) antes de renderizarlo en la interfaz, para prevenir inyecciones de código del lado del cliente u otros ataques que afecten al usuario final.

Evaluación de Riesgos de la IA

Durante todo el ciclo de vida de Desarrollo de la Inteligencia Artificial, desde su ideación hasta su puesta en producción y posterior monitoreo debe ser evaluada que cumpla y siga manteniendo los principios fundamentales de su Política de IA. El conocimientos de los riesgos así como el manejo de incidencias se vuelve crítico para la seguridad de los individuos y los sistemas.



Guidance for Risk Management of Artificial Intelligence systems - Unión Europea - AI Act
https://www.edps.europa.eu/data-protection/our-work/publications/guidelines/2025-11-11-guidance-risk-management-artificial-intelligence-systems_en

Posibles Riesgos

Principio	Riesgo	Verificaciones de impacto (ejemplo)	Posibles mitigaciones
5.1 Principio de equidad	5.1.1 Falta de calidad de datos en datos personales de entrenamiento	Evaluar si los conjuntos de datos de entrenamiento son inexactos, incompletos o contienen información mal etiquetada que pueda llevar a resultados erróneos.	1. Definir política de aseguramiento de calidad. 2. Procedimiento de evaluación de datos. 3. Auditorías regulares. 4. Técnicas estadísticas para valores atípicos. 5. Verificación humana/difusa. 6. Estandarización de formatos.
5.1 Principio de equidad	5.1.2 Sesgo en datos personales de entrenamiento	Revisar si existen errores de muestreo (datos no representativos de la población) o sesgos históricos y sociotécnicos en los datos.	1. Asegurar datos representativos (coincidencia de distribución, diversidad). 2. Seleccionar características sin sesgo. 3. Ingeniería de características (reescalado/reponderación). 4. Auditorías de sesgo. 5. Técnicas de mitigación en datos.
5.1 Principio de equidad	5.1.3 Sobreajuste a los datos personales de entrenamiento	Comprobar si el modelo ha memorizado los detalles y el ruido de los datos de entrenamiento, afectando negativamente su rendimiento con datos nuevos o no vistos.	1. Detención temprana (Early stopping). 2. Simplificación del modelo. 3. Técnicas de regularización (L1, L2, Elastic Net). 4. Abandono (Dropout).
5.1 Principio de equidad	5.1.4 Sesgo algorítmico	Analizar si el diseño del algoritmo (funciones de optimización, prevención de sobreajuste) favorece o perjudica injustamente a ciertos grupos, independientemente de los datos.	1. Algoritmos conscientes de la equidad. 2. Funciones objetivo equilibradas. 3. Auditorías de sesgo. 4. Pruebas con datos diversos. 5. Interpretabilidad del modelo. 6. Evaluar algoritmos alternativos.
5.1 Principio de equidad	5.1.5 Sesgo de interpretación	Identificar si los analistas sacan conclusiones incorrectas o sesgadas de los resultados del modelo, o si se malinterpretan las métricas de rendimiento como diagnósticos definitivos.	1. Involucrar a un equipo diverso. 2. Documentación y comunicación claras. 3. Técnicas de explicabilidad del modelo. 4. Capacitación y concientización. 5. Auditorías de interpretación.

Posibles Riesgos

Principio	Riesgo	Verificaciones de impacto (ejemplo)	Posibles mitigaciones
4 Interpretabilidad y explicabilidad	4.1 Sistema de IA ininterpretable o inexplicable	Verificar si el sistema opera como una 'caja negra' donde los procesos internos y de toma de decisiones son opacos o incomprensibles para los usuarios humanos.	1. Documentación adecuada (arquitectura, origen de datos, sesgos, limitaciones). 2. Uso de técnicas de explicabilidad (LIME o SHAP). 3. Análisis estadístico de los resultados.
5.2 Principio de exactitud	5.2.3 Salida de datos personales inexactos	Verificar si el modelo genera información incorrecta o sin sentido (alucinaciones) que no estaba presente en los datos de entrenamiento ni en la entrada.	1. Datos de entrenamiento de alta calidad. 2. Pruebas con casos límite (edge cases). 3. Datos diversos y representativos. 4. Conjunto de datos equilibrado. 5. Optimización de hiperparámetros. 6. Supervisión humana (HITL).
5.2 Principio de exactitud	5.2.4 Salida inexacta debido a la deriva de datos y deterioro de la calidad de los datos personales de entrada	Monitorear si hay cambios en las propiedades estadísticas de los datos de entrada a lo largo del tiempo (data drift) que causen predicciones inexactas.	1. Métodos de detección de deriva de datos. 2. Monitoreo de la calidad de los datos. 3. Reentrenamiento regular del modelo. 4. Ciclos de retroalimentación de los usuarios.
5.2 Principio de exactitud	5.2.5 Información poco clara del proveedor del sistema de IA	Evaluar si el proveedor de la IA proporciona información vaga o limitada sobre el funcionamiento del sistema, sus procesos de desarrollo y el origen de los datos.	1. Exigir documentación general y de la API. 2. Documentación sobre explicabilidad. 3. Documentación de ciberseguridad. 4. Prácticas de gobernanza de datos personales. 5. Procedimientos de validación y métricas de prueba.
5.3 Principio de minimización de datos	5.3.1 Recolección y almacenamiento indiscriminado	Comprobar si se están recopilando y procesando grandes volúmenes de datos personales sin criterios claros de necesidad o relevancia para el objetivo.	1. Evaluación previa de la necesidad de los datos. 2. Muestreo de datos (Data sampling). 3. Anonimización y seudonimización.

Posibles Riesgos

Principio	Riesgo	Verificaciones de impacto (ejemplo)	Posibles mitigaciones
5.4 Principio de seguridad	5.4.1 Divulgación de datos personales de entrenamiento en la salida del sistema de IA	Revisar si el modelo revela involuntariamente detalles de los individuos del conjunto de entrenamiento mediante ataques de inversión, inferencia de pertenencia o regurgitación.	1. Minimización de datos personales. 2. Técnicas de perturbación (generalización, agregación, privacidad diferencial). 3. Generación de datos sintéticos. 4. Decodificación MEMFREE.
5.4 Principio de seguridad	5.4.2 Almacenamiento de datos personales y brechas de datos personales	Verificar la vulnerabilidad de los datos de entrenamiento ante compromisos de confidencialidad o integridad (ej. envenenamiento de datos o ataques de evasión).	1. Anonimización/seudonimización. 2. Cifrado de datos en reposo. 3. Uso de datos sintéticos. 4. Prácticas de desarrollo seguro. 5. Autenticación multifactor (MFA).
5.4 Principio de seguridad	5.4.3 Fuga de datos personales a través de interfaces de programación de aplicaciones (APIs)	Analizar si las APIs exponen más datos de los previstos, carecen de controles de acceso adecuados o dejan endpoints de depuración habilitados en producción.	1. Autenticación fuerte (MFA) en APIs. 2. Control de acceso basado en roles (RBAC). 3. Limitación de tasa (Throttling). 4. Cifrado de comunicaciones (HTTPS/TLS). 5. Registro y monitoreo (SIEM). 6. Auditorías de seguridad y parcheo.
5.5 Derechos del titular de los datos	5.5.1 Acceso incompleto	Evaluar la dificultad para localizar e identificar datos personales específicos dentro de conjuntos de datos no estructurados o parámetros de modelos complejos para responder a solicitudes de acceso.	1. Conservar metadatos para facilitar la identificación. 2. Herramientas de recuperación de datos. 3. Herramientas automatizadas como MemHunter.
5.5 Derechos del titular de los datos	5.5.2 Rectificación o borrado incompleto	Comprobar si el sistema retiene patrones o información de los datos de entrenamiento que sean difíciles de aislar, rectificar o eliminar cuando un usuario ejerce sus derechos.	1. Herramientas de recuperación y rectificación. 2. Desaprendizaje automático (Machine unlearning). 3. Filtrado de salida en tiempo real.

Desafíos Riesgos de Agentes

Level	Anomaly Type	System-centric Causes	Model-centric Causes	Orchestration-centric Causes
Intra-Agent	Reasoning Anomalies	RAG DB providing noisy or outdated context.	Core model hallucination; factual incorrectness; logical fallacies.	Flawed Chain-of-Thought prompting; ambiguous instructions leading to misinterpretation.
	Planning Anomalies	Tool/API documentation is unclear, leading to impossible plans.	Model fails to decompose a complex task correctly or fails to learn how to use a new tool after several attempts.	Poor task decomposition strategy; ReAct-style prompt errors; failure to select appropriate tools.
	Action Anomalies	External API failure (e.g., 500 error); network issues; authentication failure.	Model generates incorrectly formatted parameters (e.g., wrong data type).	Agent logic fails to parse tool output correctly; incorrect mapping of intent to function call.
	Memory Anomalies	Vector DB latency or failure (long-term); infrastructure limits on context size (short-term).	"Lost in the middle" phenomenon; inherent model limits on context window.	Ineffective context compression strategy; poor RAG retrieval query generation.
Inter-Agent	Environment & Termination Anomalies	Unstable environment; resource exhaustion; API rate limiting causing loops.	Model generates an action or code (e.g., a script requesting excessive memory) that directly causes resource exhaustion or an unrecoverable state.	Agent logic enters an infinite loop; premature termination due to flawed exit conditions.
	Task Specification Anomalies	Not applicable, as the anomaly originates from user input, not system state.	Model misinterprets an ambiguous user request instead of asking for clarification.	The initial prompt from the user is vague, incomplete, or contradictory.
	Security & Trust Anomalies	Compromised external tools (Tool Poisoning); insecure communication channels.	Model is susceptible to jailbreaking or prompt injection attacks.	Flawed agent interaction protocols that can be exploited.
	Communication & Emergent Anomalies	Network latency or partitioning in multi-agent systems.	Not applicable, as this is a system-level interaction issue.	Inefficient communication protocols; flawed multi-agent coordination logic leading to negative emergent behavior.



Despliegue y monitoreo

Despliegue - Subir a producción

Deploy: la tarea de subir / poner nuestro código en uso para los usuarios o consumidores del sistema.

Deployar un sistema es una tarea riesgosa, y desplegar un sistema de ML en producción lo es aún más.

En ML, varias cosas pueden ir mal:

- Fallas, bugs, errores, páginas o APIs que no responden o accesos restringidos
- Servidores sobrecargados de pedidos, respuesta lenta (baja latencia)
- Errores funcionales, no hace lo que se espera que haga
- Errores probabilísticos, para algunos usuarios, las respuestas no son satisfactorias

En la práctica, casi ningún sistema es disponibilizado para la totalidad de sus consumidores o los deploys se hacen de manera controlada. Casualmente, estas maneras controladas de hacer deploys pueden servir para testear en forma online el sistema de AI



Adquisición de Infraestructura para el despliegue

- **Infraestructura contratada propietaria:** donde la empresa/laboratorio hace el mantenimientos de los equipos y del software específico necesario: por ejemplo, tenes un servidor en la red con bases de datos MySQL con distintos accesos y permisos.
- **La nube:** distintos proveedores de cloud que se ocupan del mantenimiento y seguridad de los datos. Estos proveen diferentes servicios para el almacenamiento y trackeo de data estructurada y no estructurada. También ofrecen integraciones con diferentes almacenamientos de datos: bases de datos de renombre, NoSQL, etc. Las nubes incluso ofrecen servicios de baja latencia para grandes datos.
- Esto refiere a dos **modelos de negocios CapEx y OpEx**. Estos modelos hablan sobre la inversión en capital de la infraestructura vs. el costo operacional de usar los servicios de la nube - modelo “pago por uso”.

Escalabilidad

La infraestructura tiene que ser capaz de agregar más equipos (upscaling) o remover equipos en desuso (downscaling). Esto requiere el monitoreo del equipo y en caso de ser posible, configuraciones de escalamiento automático a partir de métricas. Por ejemplo, cantidad de requests por segundo (solicitudes web http).

Por otro lado, el escalamiento se puede hacer de dos maneras: vertical u horizontal.

- Escalamiento Horizontal: agregar más equipos de las mismas características para contener el incremento de tráfico o recursos requeridos.
- Escalamiento Vertical: aumentar las capacidades de los equipos actuales por ejemplo, agregando más memoria o mejorando los procesadores.



Vertical Scaling

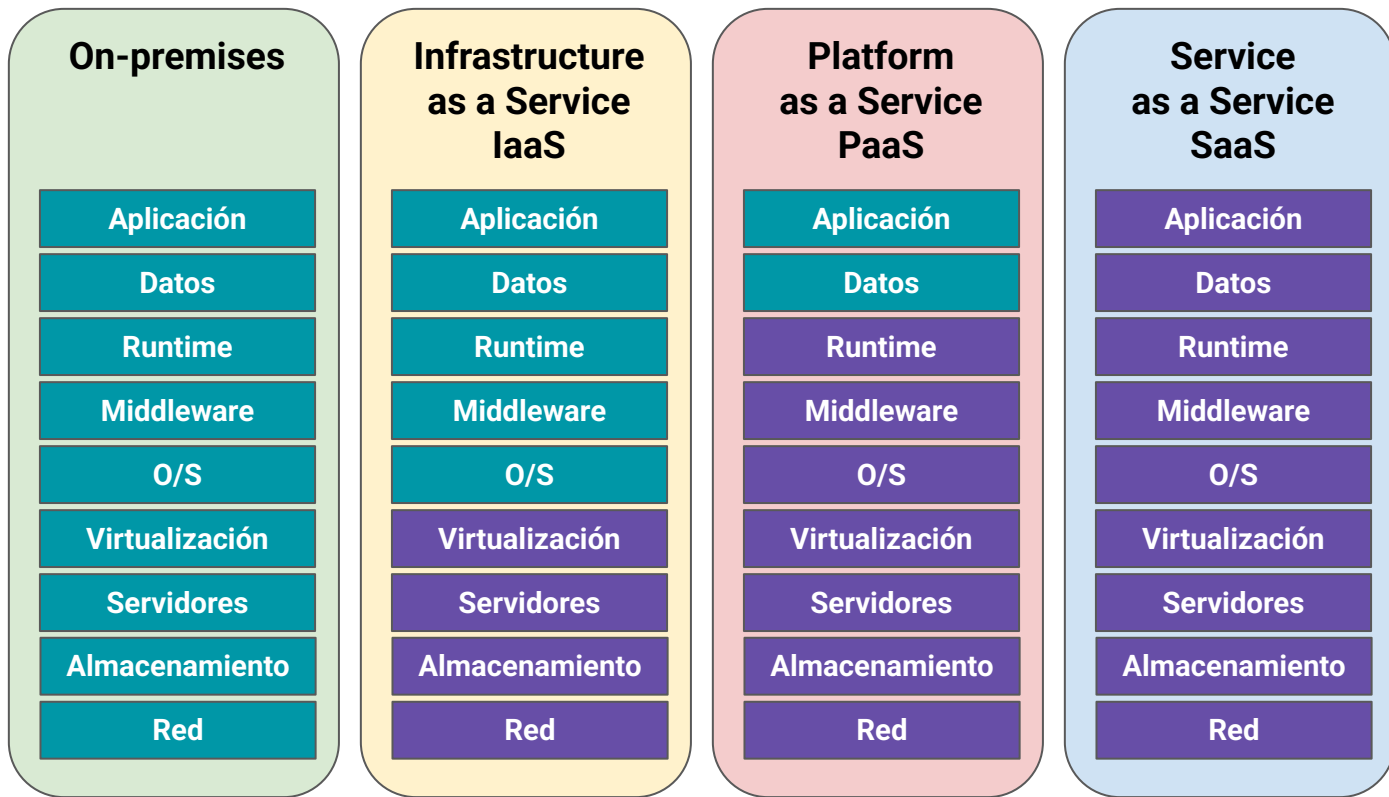
(Scaling up)



Horizontal Scaling

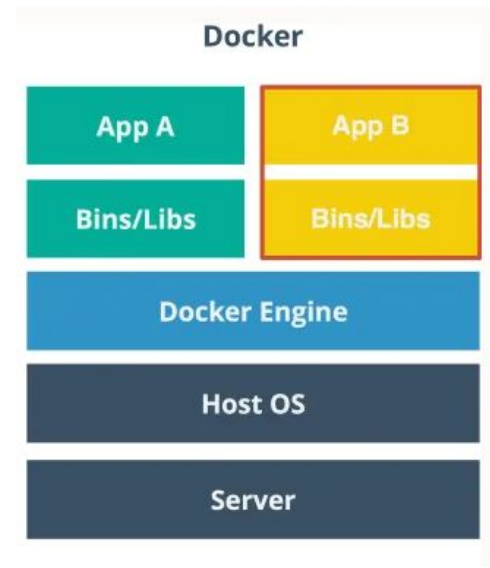
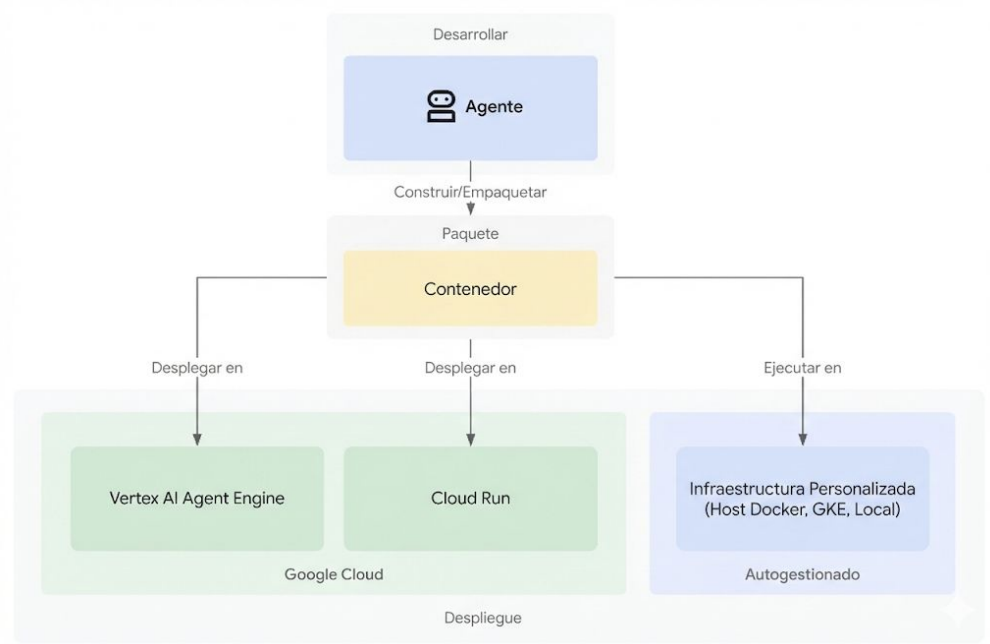
(Scaling out)

On-premises, SaaS, PaaS, IaaS en la nube

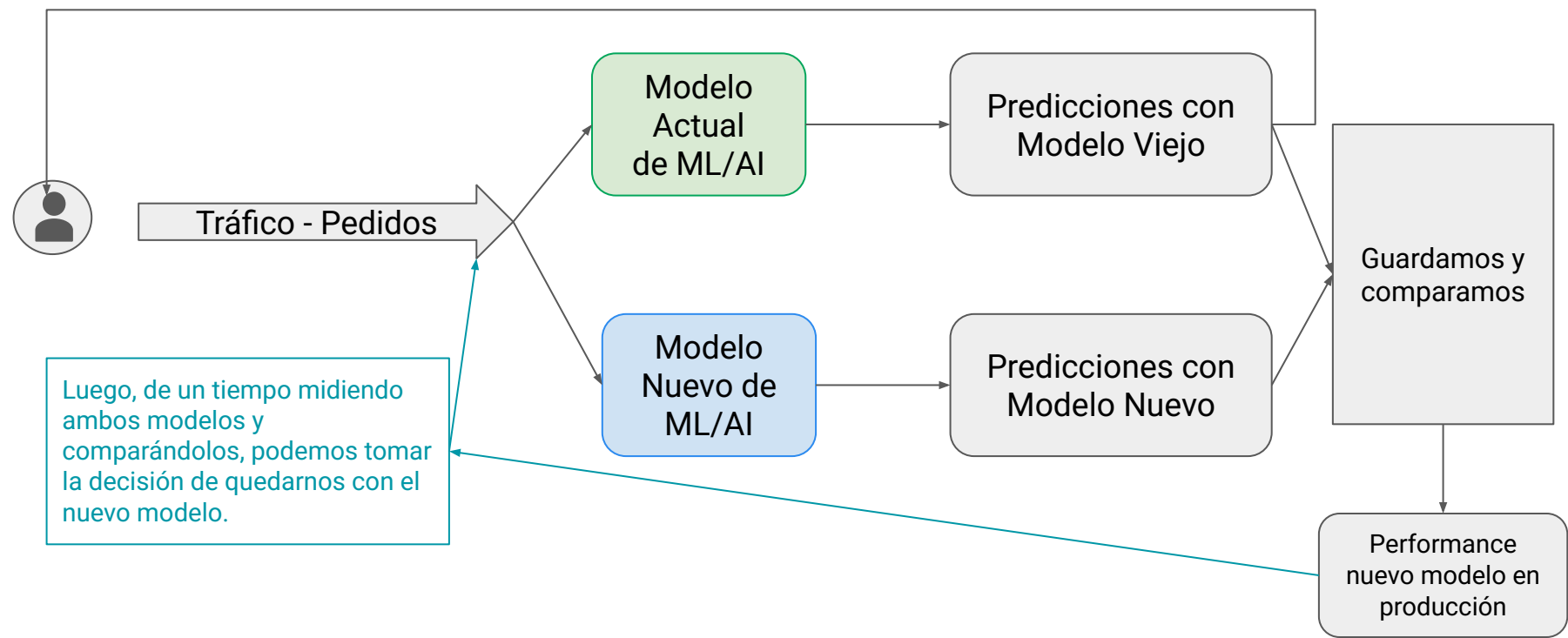


Contenedores

Con respecto a la infraestructura, podemos desplegar en la Nube en los servicios de Google Cloud o cualquier otro proveedor o infraestructura que tengamos que pueda servir o ejecutar el agente. Una buena práctica es usar Contenedores como Docker.



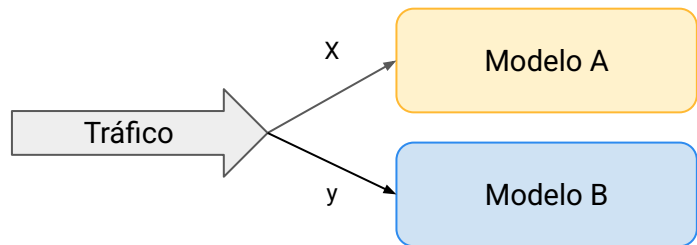
Shadow Deployment para el modelo



Lo mismo aplicaría para el sistema si estamos cambiando el sistema de AI entero

A/B Testing

A/B Testing es una forma de comparar dos variantes de un objeto, generalmente es una porción de software, un email, de alguna forma que nos permita decidir cuál es más efectiva.



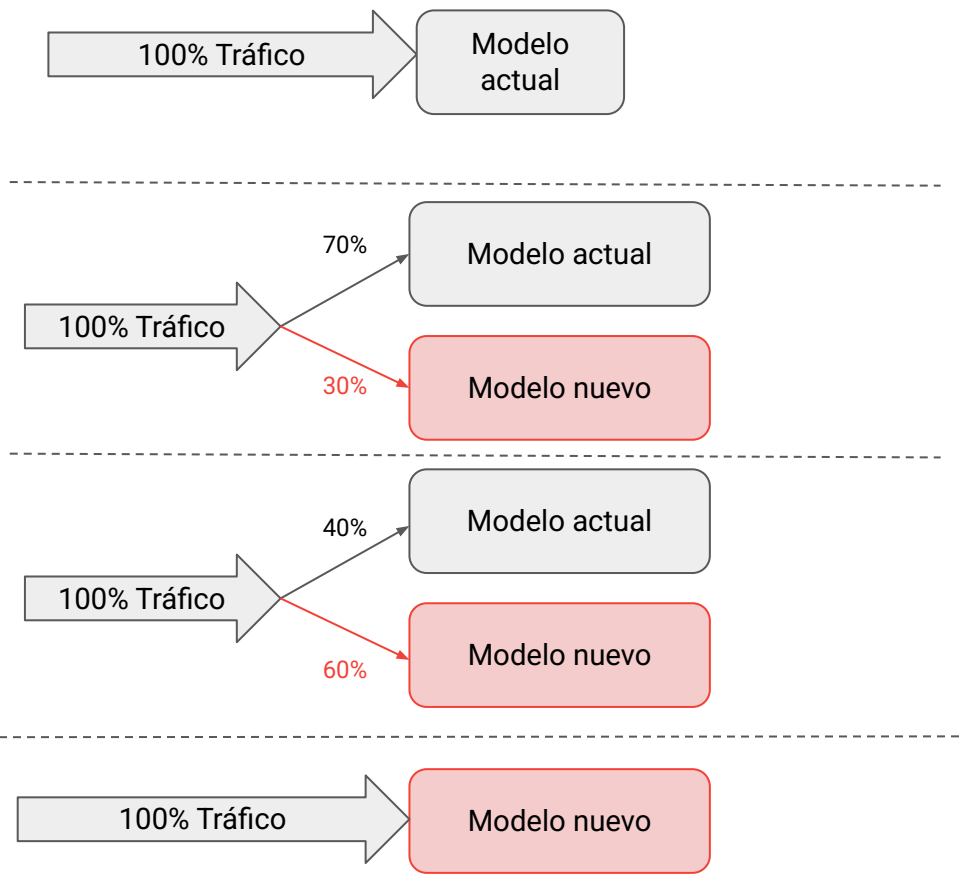
x, y son porciones aleatorias del tráfico
disjuntas

Probamos ambos modelos a ver cuál
funciona mejor mandando la misma
cantidad de tráfico de manera aleatoria
a cada uno y viendo las métricas



Analizamos las predicciones de
ambos modelos y el feedback del
usuario, decidimos cuál vamos a
usar.

Canary Deploy



Gradualmente se transfiere el tráfico al nuevo modelo.

Se va redirigiendo el tráfico de a poco y realizando verificaciones al nuevo modelo. Por ejemplo, la primera semana, se manda el 30% del trafico al nuevo modelo. Una vez que se observa que todo funciona como se espera, a semana siguiente, o más tarde, se manda el 60%. En incrementos secuenciales y validando el funcionamiento en cada paso.

Si los resultados no son los esperados, se vuelve al modelo actual.

El tamaño de los incrementos y el momento depende de las características del problema y la decisión que toma el equipo de ingenieros.

Monitoreo

Monitoreo es el acto de traquear, medir y loguear las diferentes métricas que nos pueden ayudar a ver las cosas que pueden salir mal. Generalmente, esa información puede ser usada para investigar que falló o encontrar algún error, a esto se lo denomina observabilidad.

La información de logueo y performance lo de los diferentes niveles son ampliamente usadas en la construcción de software.

Suceden a distintos niveles:

- Equipamiento - CPU - GPU - Memoria - Discos
- Red - Paquetes enviados
- Servidor Web - requests / solicitudes
- Aplicación - Logueo de determinados eventos, funciones

Tenemos que considerar también monitorear métricas de ML

Logs

Logs

```

GNU nano 2.0.9                               File: access_log

192.168.1.4 - - [09/May/2014:09:34:25 -0400] "GET / HTTP/1.1" 200 100 "-" "Mozilla/5.0 (Windows NT 6.0; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0"
192.168.1.4 - - [09/May/2014:09:34:25 -0400] "GET /favicon.ico HTTP/1.1" 404 288 "-" "Mozilla/5.0 (Windows NT 6.0; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0"
192.168.1.4 - - [09/May/2014:09:40:53 -0400] "GET /admin HTTP/1.1" 301 310 "-" "-"
192.168.1.4 - - [09/May/2014:09:40:53 -0400] "GET /admin/ HTTP/1.1" 200 103 "-" "-"
192.168.1.4 - - [09/May/2014:09:40:53 -0400] "GET /favicon.ico HTTP/1.1" 404 288 "-" "-"
192.168.1.4 - - [09/May/2014:09:50:18 -0400] "GET /admin/ HTTP/1.1" 401 478 "-" "-"
192.168.1.4 - - torkd [09/May/2014:09:51:05 -0400] "GET /admin/ HTTP/1.1" 401 478s "-" "-"
192.168.1.4 - - tofjk [09/May/2014:09:51:20 -0400] "GET /admin/ HTTP/1.1" 200 103 "-" "-"
192.168.1.4 - - - [09/May/2014:10:24:19 -0400] "GET /admin HTTP/1.1" 401 478 "-" "-"
192.168.1.4 - - [09/May/2014:10:25:34 -0400] "GET /favicon.ico HTTP/1.1" 404 288 "-" "-"
192.168.1.4 - - [09/May/2014:10:27:31 -0400] "GET /admin HTTP/1.1" 404 280 "-" "-"
192.168.1.4 - - [09/May/2014:10:27:32 -0400] "GET /admin HTTP/1.1" 404 280 "-" "-"
192.168.1.4 - - [09/May/2014:10:27:38 -0400] "GET / HTTP/1.1" 200 100 "-" "Mozilla/5.0 (Windows NT 6.0; WOW64; rv:10.0) Gecko/20100101 Firefox/10.0"
192.168.1.4 - - [09/May/2014:10:30:44 -0400] "GET /admin HTTP/1.1" 301 310 "-" "-"
192.168.1.4 - - [09/May/2014:10:30:44 -0400] "GET /admin/ HTTP/1.1" 200 103 "-" "-"
192.168.1.4 - - [09/May/2014:10:36:33 -0400] "GET /admin/ HTTP/1.1" 401 478 "-" "-"
192.168.1.4 - - kflfl [09/May/2014:10:36:53 -0400] "GET /admin/ HTTP/1.1" 401 478s "-" "-"
192.168.1.4 - - torkd [09/May/2014:10:37:08 -0400] "GET /admin/ HTTP/1.1" 200 103 "-" "-"
192.168.1.4 - - [09/May/2014:11:07:18 -0400] "GET /admin HTTP/1.1" 401 478 "-" $

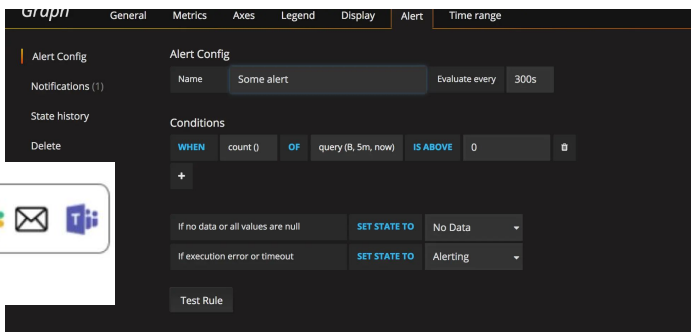
^G Get Help      ^O WriteOut     ^R Read File    ^V Prev Page    ^U Cut Text     ^C Cur Pos
^X Exit          ^J Justify      ^W Where Is    ^Y Next Page   ^U UnCut Text   ^T To Spell

```

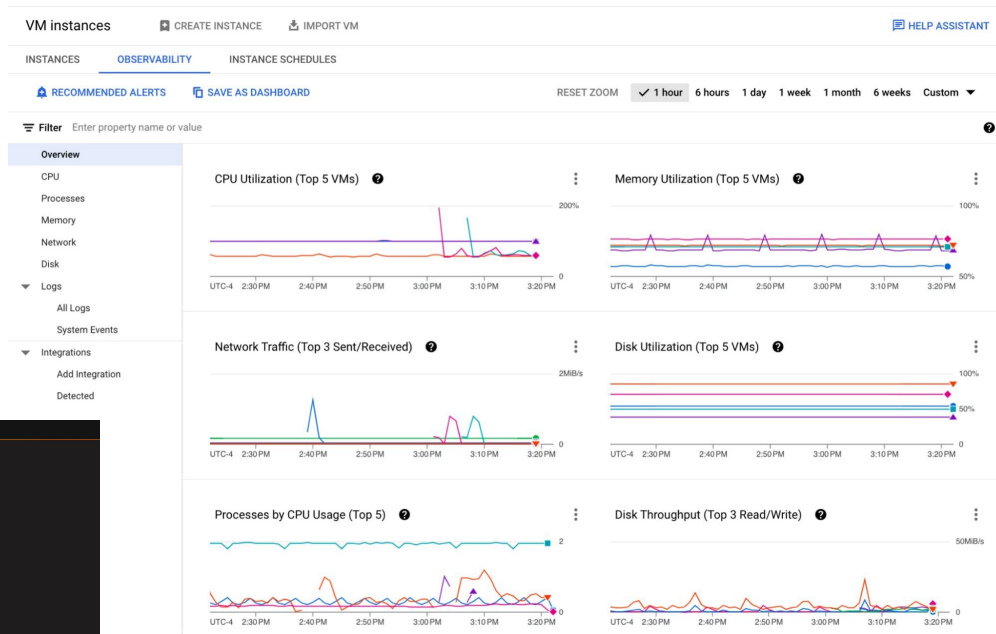
Alertas



**Support/Ops
manager**



Dashboards



Monitoreo - Logging

En el código, podemos incluir logueo explícito que quedará almacenado en archivos de texto (logfiles) usando la librería de python logging

```
import logging

logging.basicConfig(
    level=logging.DEBUG,
    format='%(asctime)s - %(levelname)s - %(name)s - %(message)s'
)

# Tu código de agente ADK sigue...
# from google.adk.agents import LlmAgent
# ...
```

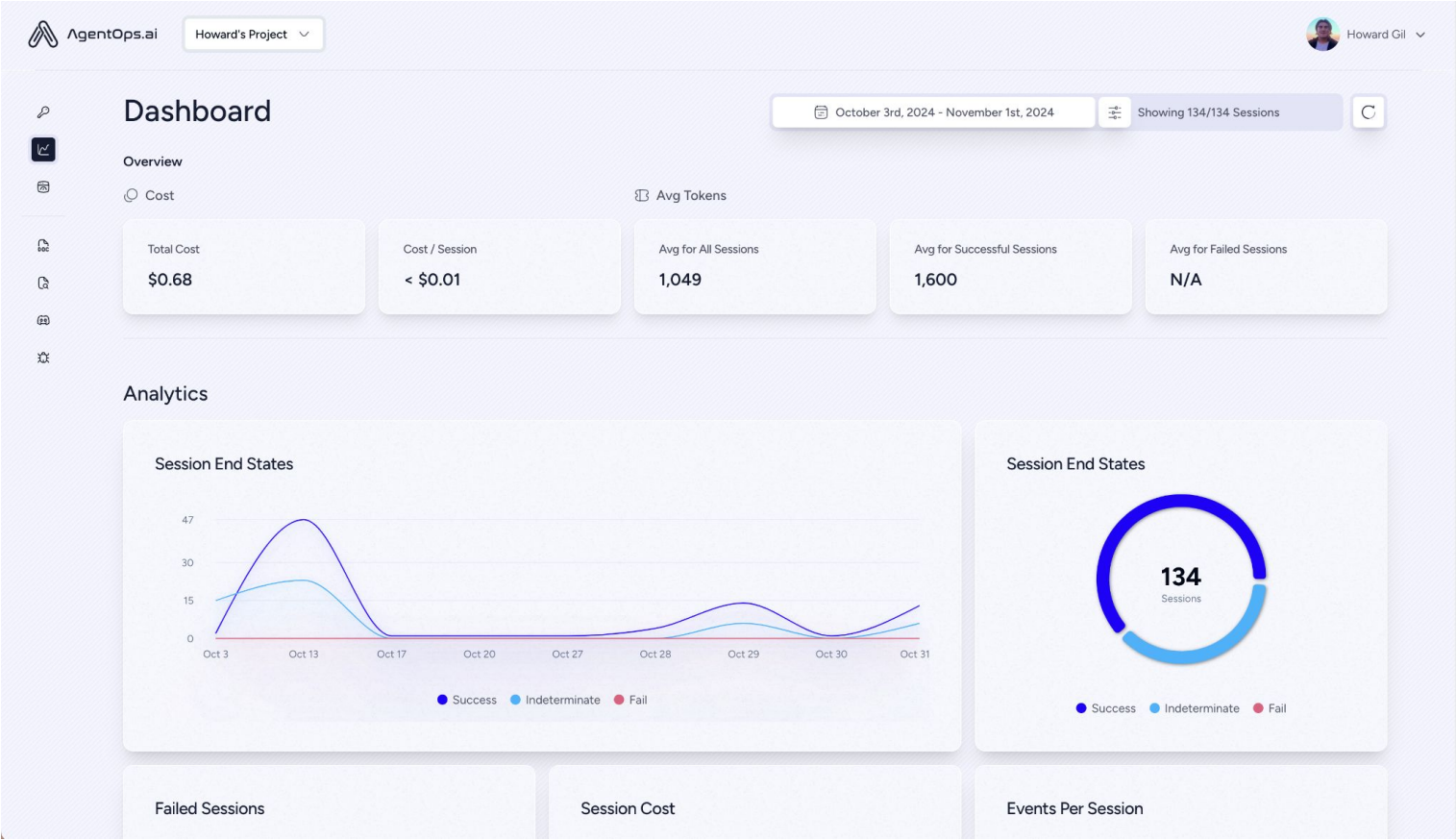
```
2025-07-08 11:22:33,456 - DEBUG - google_adk.google.adk.models.google_llm - LLM Request: contents
```

Segmento de Registro	Especificador de Formato	Significado
2025-07-08 11:22:33,456	%(asctime)s	Marca de tiempo
DEBUG	%(levelname)s	Nivel de severidad
google_adk.models.google_llm	%(name)s	Nombre del registrador (el módulo que produjo el registro)
LLM Request: contents { ... }	%(message)s	El mensaje de registro real

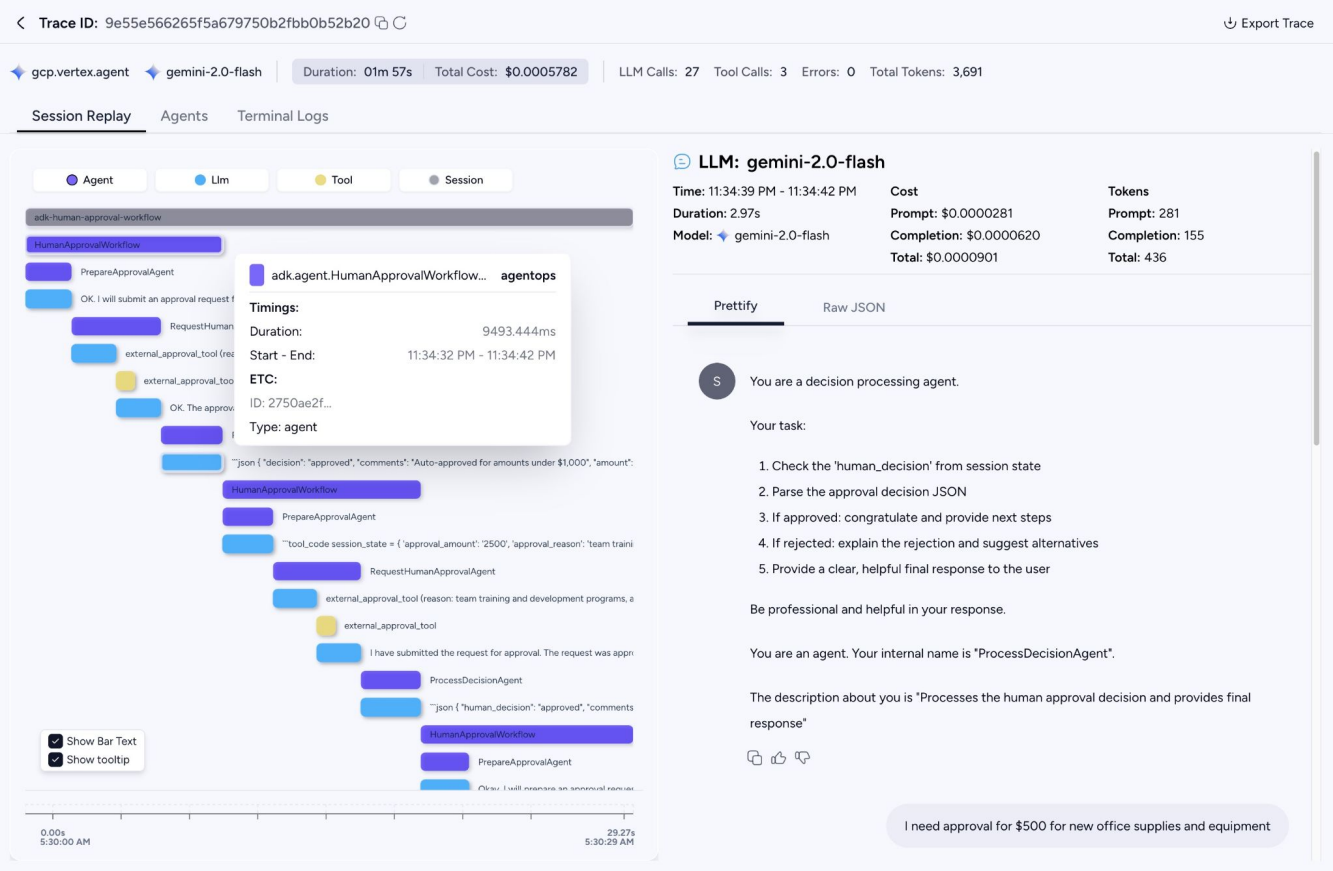
<https://adk-es.fabian-castro-c.dev/observability/logging/#configurando-el-registro-con-la-cli-de-adk>

Nivel	Descripción	Tipo de Información Registrada
DEBUG	Crucial para la depuración. El nivel más verboso para información de diagnóstico detallada.	- Prompts Completos del LLM: La solicitud completa enviada al modelo de lenguaje, incluyendo instrucciones del sistema, historial y herramientas. - Respuestas detalladas de API de servicios. - Transiciones de estado interno y valores de variables.
INFO	Información general sobre el ciclo de vida del agente.	- Inicialización y arranque del agente. - Eventos de creación y eliminación de sesiones. - Ejecución de una herramienta, incluyendo su nombre y argumentos.
WARNING	Indica un problema potencial o uso de características obsoletas. El agente continúa funcionando, pero puede requerir atención.	- Uso de métodos o parámetros obsoletos. - Errores no críticos de los que el sistema se recuperó.
ERROR	Un error grave que impidió que una operación se completara.	- Llamadas de API fallidas a servicios externos (por ejemplo, LLM, Servicio de Sesión). - Excepciones no manejadas durante la ejecución del agente. - Errores de configuración.

Monitoreo de Agentes



Monitoreo de Agentes



Métricas de Negocio

- Métricas de ML son necesarias: f1-score, recall, accuracy, precision, otras.
- También, es importante demostrar el impacto que pueden generar en el negocio y esto se hace a través de distintas métricas de negocio.
- Las Métricas de Negocio están relacionadas al problema a resolver y pueden ser más técnicas o más orientada a los negocios. Muchas incluso llegan a los altos niveles ejecutivos. Algunos ejemplos de páginas web o marketing:
 - Impressions - impresiones
 - CTR -- Click Through Rate - clicks rate: Por ejemplo, en artículos recomendados por una AI, cuantos reciben click
 - Conversion Rate - ratio de conversión: Por ejemplo, en artículos recomendados por una AI, cuantos se transforman en venta
 - ROI - Retorno de la inversión

MLOps

MLOps (Machine Learning Operations) es un paradigma que incluye aspectos como las mejores prácticas, conjuntos de conceptos y una cultura de desarrollo en lo que respecta a la conceptualización, implementación, monitoreo, despliegue y escalabilidad de extremo a extremo de productos de aprendizaje automático. Sobre todo, es una práctica de ingeniería que aprovecha tres disciplinas contribuyentes: aprendizaje automático, ingeniería de software (especialmente DevOps) e ingeniería de datos. MLOps tiene como objetivo producir sistemas de aprendizaje automático al cerrar la brecha entre el desarrollo (Dev) y las operaciones (Ops). Básicamente, MLOps tiene como objetivo facilitar la creación de productos de aprendizaje automático al aprovechar estos principios: automatización de CI/CD, orquestación de flujo de trabajo, reproducibilidad; control de versiones de datos, modelos y códigos; colaboración; capacitación y evaluación continuas de ML; seguimiento y registro de metadatos de ML; monitoreo continuo; y bucles de retroalimentación.

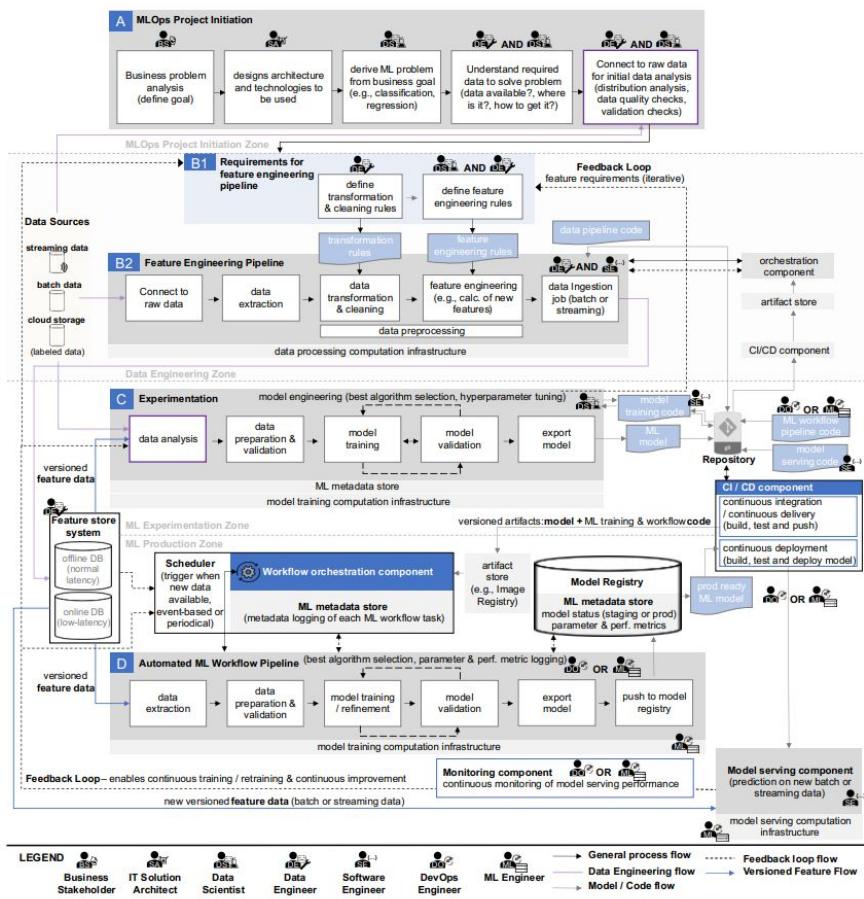
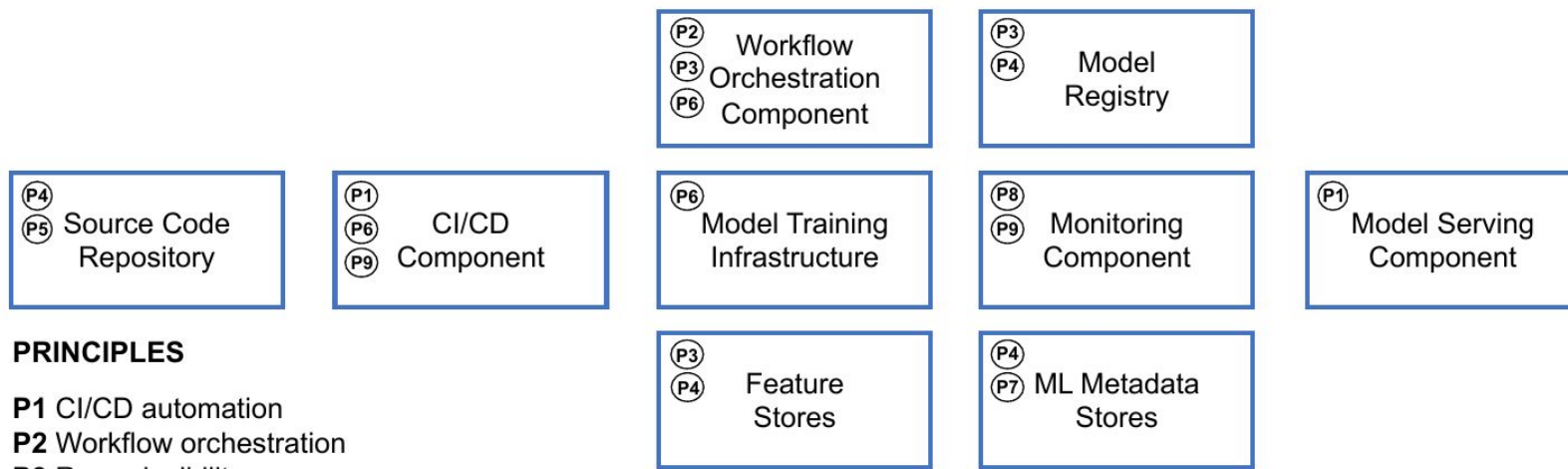


Figure 4. End-to-end MLOps architecture and workflow with functional components and roles

MLOps - Principios y Componentes



PRINCIPLES

- P1** CI/CD automation
- P2** Workflow orchestration
- P3** Reproducibility
- P4** Versioning of data, code, model
- P5** Collaboration
- P6** Continuous ML training & evaluation
- P7** ML metadata tracking
- P8** Continuous monitoring
- P9** Feedback loops

COMPONENT

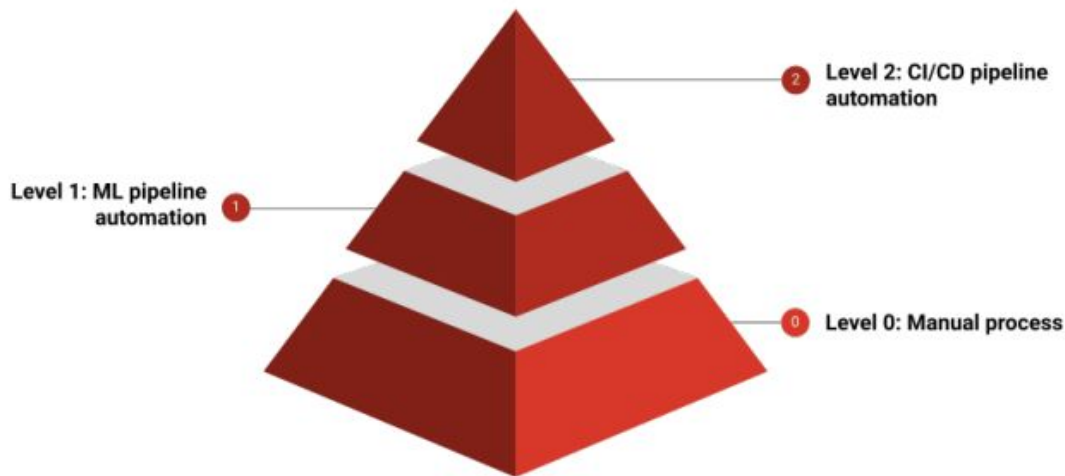
Machine Learning Operations (MLOps): Overview, Definition, and Architecture

<https://arxiv.org/pdf/2205.02302>

Niveles de Maduración de MLOps

Acorde al grado de automatización / maduración de los distintos pipelines se definen niveles de maduración:

- Nivel 0:
Procesamiento Manual
- Nivel 1:
ML Pipeline automatizado
- Nivel 2:
CI/CD pipeline automatizado



- Los proveedores de cloud pueden definir otros niveles: [Symeonidis et al. MLOps - Definitions, Tools and Challenges, 2022](#)
- Hay trabajos que buscan definirlos: [John et al., Towards MLOps: A Framework and Maturity Model, 2021](#)

LLMOps

Se la considera una subcategoría del MLOps especializada en el deployment de Modelos de Lenguajes.

LLMOps apunta a resolver los problemas intrínsecos al deployment de Modelos de Lenguaje.

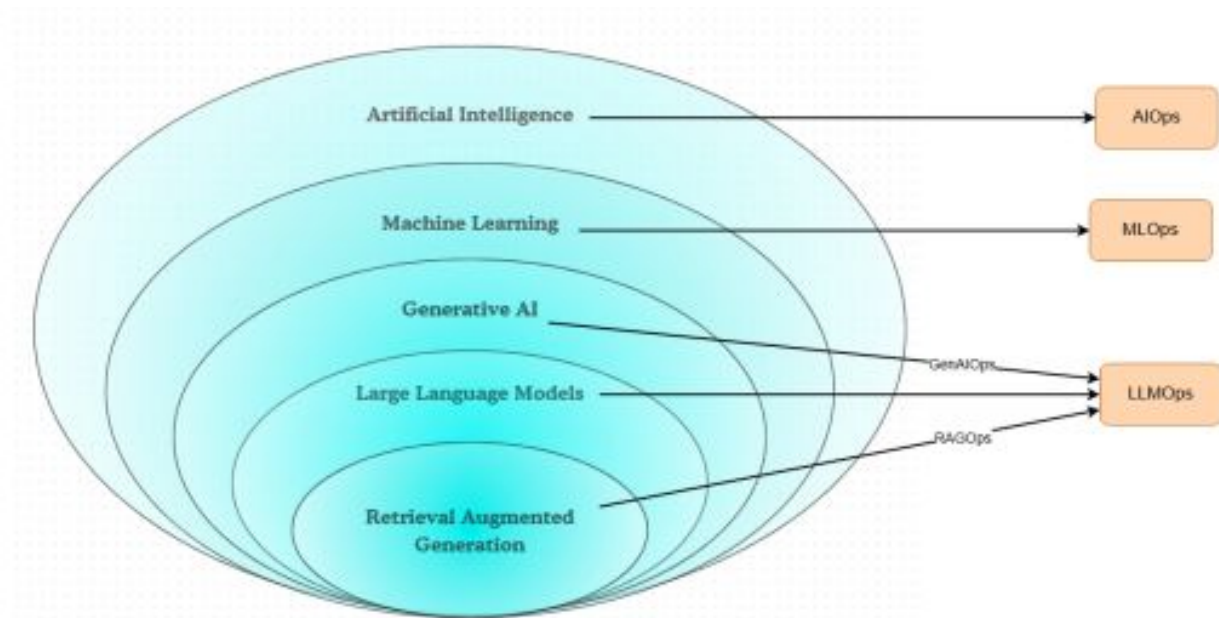


Figure 1. A hierarchy of AI types and associated Ops organized by the level of specialization [57].

Transitioning from MLOps to LLMOps: Navigating the Unique Challenges of Large Language Models

LLMOps

LLMOps involucra una serie de pasos diferentes, incluyendo:

- **Recopilación y preparación de datos:** Los LLM requieren grandes cantidades de datos para entrenarse. Estos datos deben ser recopilados y preparados de una manera que sea adecuada para el entrenamiento del modelo.
- **Desarrollo del modelo:** Los LLM se desarrollan utilizando una variedad de técnicas, incluyendo el aprendizaje no supervisado, el aprendizaje supervisado y el aprendizaje por refuerzo.
- **Despliegue del modelo:** Una vez que se ha desarrollado un LLM, debe implementarse en un entorno de producción. Esto implica configurar la infraestructura necesaria y preparar el modelo para ejecutarse en una plataforma específica.
- **Gestión del modelo:** Los LLM requieren una gestión continua para garantizar que funcionen como se espera. Esto incluye monitorear el rendimiento del modelo, volver a entrenarlo según sea necesario y asegurarse de que sea seguro.

Mejores prácticas



Transitioning from MLOps to LLMOps: Navigating the Unique Challenges of Large Language Models

AgentOps

AgentOps es una nueva disciplina —basada en los principios de DevOps y MLOps— dedicada a gestionar, optimizar y supervisar todo el ciclo de vida de los agentes autónomos de Inteligencia Artificial.

Las fases clave para aplicar estas prácticas son:

- **Desarrollo:** Definición de objetivos, restricciones y diseño de los flujos de datos del agente.
- **Pruebas:** Evaluación previa al lanzamiento dentro de un entorno seguro y simulado (*sandbox*).
- **Monitoreo:** Supervisión del rendimiento, latencia y llamadas a las API una vez que el agente está en producción.
- **Retroalimentación:** Uso de herramientas para registrar errores y aplicar mejoras continuas basadas en la interacción del usuario.
- **Gobernanza:** Implementación de políticas y límites éticos para garantizar que el agente cumpla con las normativas legales (como la Ley de IA de la UE)

<https://www.ibm.com/think/topics/agentops>

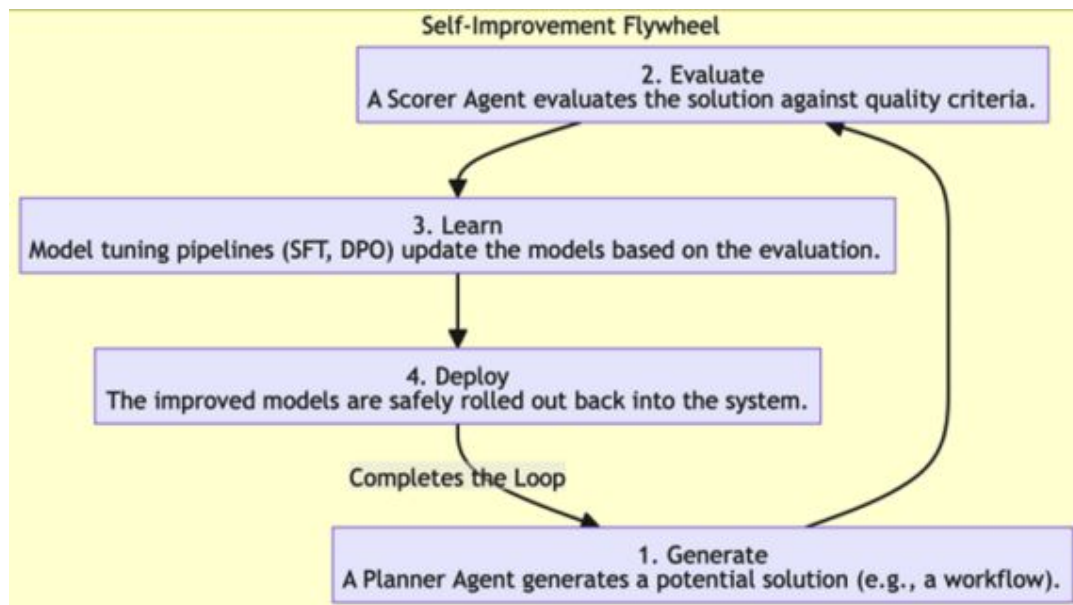
Agentes que aprenden: Ciclo Virtuoso de Automejora

El **Ciclo Virtuoso de Auto-Mejora** (*Self-Improvement Flywheel*) es un modelo conceptual que ilustra este ciclo continuo de cuatro etapas:

- **Generar (Generate):** El sistema genera una solución potencial, como un flujo de trabajo de múltiples pasos (*multi-step workflow*), un fragmento de código o un análisis complejo.

Para garantizar una mejora a largo plazo, esta etapa debe equilibrar la **explotación** (*exploitation* — refinar estrategias conocidas y efectivas) con la **exploración** (*exploration* — intentar enfoques novedosos). Sin exploración, el sistema corre el riesgo de estancarse en un **óptimo local** (*local optimum*), limitándose a repetir sus patrones establecidos en lugar de descubrir soluciones superiores.

- **Evaluar (Evaluate):** La solución generada se evalúa críticamente frente a un conjunto de criterios de calidad (ej., corrección algorítmica, eficiencia, seguridad, exactitud factual o adherencia a reglas de negocio). Este es el paso más crucial, ya que un sistema agéntico solo puede mejorar aquello que es capaz de medir.
- **Aprender (Learn):** Basándose en la evaluación, los modelos subyacentes del sistema se actualizan o se ajustan finamente (*fine-tuned*) para maximizar la probabilidad de producir salidas de alta calidad en el futuro.
- **Desplegar (Deploy):** Los modelos mejorados se despliegan de forma segura nuevamente en el sistema en producción, haciendo que todo el ecosistema tenga mayores capacidades para afrontar el siguiente ciclo.



Manejo de Costos y "Tokenomics" (economía de tokens)

